

CoachApp — دليل تسليم المطور

الجمهور: مطور يستلم ال backend. تشرح هذه الوثيقة التقنيات والمعمارية وكل كيان (Entity) ونموذج المصادقة/العزل بين المستأجرين (Tenancy) وخدمات جهة المدرب وجهة المتدرب والقواعد الشاملة وكيفية البناء/التشغيل/التوسعة.

القاعدة الذهبية لهذا الكود: *الكود هو المرجع. * عند اختلاف أي وثيقة مع الكود، الكود يفوز. الشريحة المرجعية التي تُحتذى هي **Trainee / WorkoutPlan** تحت **src/**.

ملاحظة: تُبقي هذه الوثيقة أسماء الكلاسات والملفات والمسارات والمعرفات البرمجية بالإنجليزية لأنها كود حرفي.

1. التقنيات والشكل العام

- **الإطار:** ABP Framework 10.4.0 على **NET 10**.
- **القالب:** قالب ABP للتطبيق الواحد (**app**) — **وليس** القالب المعياري (**modular**). **لا يوجد مجلد **modules/**؛ كل شيء تحت **src/**.
- **النمط:** شرائح عمودية (vertical slices) عملية بأسلوب **مجلّدات المزايا** (على غرار **../JasimExpress/express_api**).
- **قاعدة البيانات:** SQL Server، والمخطط يُدار عبر **EF Core migrations** (لا SQL يدوي أبداً).
- **المصادقة:** OpenIddict (خادم مصادقة + تحقق من التوكن في نفس عملية الاستضافة).
- **تعدّد المستأجرين (Multi-tenancy):** مُفعّل — **مدرب واحد = مستأجر واحد**.
- **التحويل (Mapping):** **Mapster** (لا AutoMapper/Mapperly).
- **ال API:** متحكّمات تلقائية (**conventional controllers**) من ABP — تُنشر AppServices كـ REST تلقائياً؛ لا متحكّمات مكتوبة يدوياً.
- **مُستبعد عمداً:** MediatR/CQRS، AutoMapper، Mapperly، FluentValidation، مستودعات مخصّصة لـ CRUD.

البسيط، وكلاسات ***Factory**/***Manager** لـ CRUD البسيط.

⚠ **مفارقة في التسمية:** مقطع المجلد وفضاء الأسماء مكتوب **Entites** (بحرف **i** ناقص). هذا مقصود وثابت في كل مكان — الكيان + DTOs + الواجهة تحت **CoachApp.Entites.<Feature>**. لا

2. تخطيط الحلّ (Solution)

المشروع	المسؤولية
<code>CoachApp.Domain.Shared</code>	الثوابت (<code>Entites/<Feature>/</code>)، التعدادات (<code>Enums/</code>)، <code>CoachAppDomainErrorCodes</code> ، الأدوار/البادئات، الترجمة (<code>Localization/CoachApp/*.json</code>)
<code>CoachApp.Domain</code>	الكيانات (<code>Entites/<Feature>/</code>)، بذور البيانات (seeders)، بذرة <code>OpenIddict</code> ، حارس المدرّب الواحد
<code>CoachApp.Application.Contracts</code>	<code>I<Entity>AppService</code> + DTOs (<code>Entites/<Feature>/</code>)، الصلاحيات (<code>Permissions/</code>)
<code>CoachApp.Application</code>	<code>AppServices</code> (<code>Apis/<Feature>/</code>)، <code>CoachAppMapsterConfig</code> ، <code>MyTraineeAppServiceBase</code> ، <code>MapsterObjectMapper</code>
<code>CoachApp.EntityFrameworkCore</code>	<code>CoachAppDbContext</code> ، <code>EntityConfigurations/<Feature>/</code> ، <code>Migrations/</code>
<code>CoachApp.HttpApi</code> <code>.HttpApi.Host</code>	المتحكّمات التلقائية، ربط الاستضافة، Swagger، المصادقة
<code>CoachApp.DbMigrator</code>	تطبيق كونسول: يطبّق الـ migrations ويزرع الأدوار/الصلاحيات/المشرف

اصطلاح فضاء الأسماء (يُسقط اسم الطبقة)

- `CoachApp.Entites.<Feature>` → الكيان + DTOs + `I<Entity>AppService`
- `CoachApp.Apis.<Feature>` → `AppService`
- `CoachApp.Enums` → التعدادات
- `CoachApp.EntityConfigurations.<Feature>` → إعداد EF

- `CoachApp.Permissions` → الصلاحيات
- رموز الأخطاء `CoachApp` → `(CoachAppDomainErrorCodes)`
- `<Feature>` هو اسم الكيان بصيغة الجمع (`WorkoutPlans`، `Trainees`...).

3. كيف يتدقق الطلب

HTTP طلب

- `AppService` (مولّد من واجهة الـ) التلقائي ABP متحكّم
- (على مستوى الكلاس + لكل ميثود) `[Authorize]` فحص صلاحية
- `AppService (Apis/<Feature>)` - "الطبقة" السميّة -
 - `ICurrentUser / ICurrentTenant` تحديد المستخدم/المستأجر الحالي
 - `IRepository<T, Guid>.GetQueryableAsync() + WhereIf + Dynamic.Core` استعلام عبر
 - `OrderBy + Skip/Take + AsyncExecuter`
 - `BusinessException(errorCode)` فحوص وجود/فراة عبر التجميعات → رمي
 - `static Create(...)` / `Add*`, `Clear*`, `Activate...` ميثودات السلوك
 - `IRepository<T, Guid>` الحفظ عبر
 - `Mapper.Map` (يُمرّ عبر) `ObjectMapper.Map` → `DTO` تحويل الكيان
 - بيانات عرض عبر-تجميعية (أسماء التمارين/الأطعمة، مجاميع `DTOS` إثراء
- (الماكروز)
- `DTO` استجابة (تُسلّسَل تلقائيًا)

يُطبّق تعدّد المستأجرين في الـ `pipeline` (`UseMultiTenancy`، بعد المصادقة)، فتُصقّى كل استعلامات `IMultiTenant` تلقائيًا للمستأجر الحالي — نادرًا ما تصقّى بـ `TenantId` يدويًا.

4. نموذج المجال (Domain)

4.1 اصطلاحات عامّة عبر كل الكيانات

- الجذور التجميعية (**Aggregate roots**): `IMultiTenant + FullAuditedAggregateRoot<Guid>`
- `(public virtual Guid? TenantId)`. تمنح `Id`، تدقيق الإنشاء/التعديل، الحذف الناعم (**soft delete**)، وعزل المستأجر.
- الكيانات الابنة (المملوكة): `Entity<Guid>`. مفتاحها الخلفي للأب `protected set` (يُضبط فقط عبر الباني). تُنشأ حصريًا عبر ميثودات `Add*` في الجذر — وهذا هو حدّ التجميعية.
- التحقق: عبر `Check.Length / Check.NotNullOrWhiteSpace` من ABP مقابل الـ `*Consts` المشتركة

(يرمي `ArgumentException`). الكيانات نفسها لا ترمي `BusinessException` — القواعد الحقيقية عبر التجميعات (خطة مفصلة واحدة، "حدّ 100 مُدخّل"، وجود المراجع) تُفرض في طبقة التطبيق.

- **المصنع + السلوك:** لكل جذر `static Create(Guid id, ..., Guid? tenantId = null)`؛ وتغييرات الحالة

عبر ميثودات سلوك (`Add*`، `Clear*`، `Deactivate/Activate`). لا `*Manager/Factory`.

4.2 خريطة التجميعات

Trainee	(مستقل — <code>UserId</code> عبر <code>IdentityUser</code> جذر؛ يرتبط بـ)
Exercise	(مستقل — جذر؛ مكتبة تمارين المدرب)
Food	(مستقل — جذر؛ مكتبة أطعمة المدرب)
WorkoutPlan	(<code>IsActive</code> ، جذر، <code>TraineeId</code>)
└ WorkoutDay	(<code>ScheduledDay: DayOfWeek?</code> ، <code>Order</code>)
└└ WorkoutExercise	(<code>Exercise</code> ، <code>Sets/Reps/WeightKg/RestSeconds/Notes</code>)
NutritionPlan	(، أهداف ماكروز يومية اختيارية <code>IsActive</code> ، جذر، <code>TraineeId</code>)
└ Meal	(<code>Order</code>)
└└ MealItem	(<code>Food</code> ، <code>Quantity</code> = عدد الحصص)
WorkoutPlanTemplate	(مخطط قابل لإعادة — <code>IsActive</code> / جذر؛ بلا <code>TraineeId</code>)
└ WorkoutTemplateDay	(يُنقل إلى النسخ <code>ScheduledDay</code>)
└└ WorkoutTemplateExercise	(<code>Exercise</code>)
NutritionPlanTemplate	(؛ الأهداف تُنقل إلى النسخ <code>IsActive</code> / جذر؛ بلا <code>TraineeId</code>)
└ NutritionTemplateMeal	
└└ NutritionTemplateItem	(<code>Food</code>)
WorkoutLog	(اختياريان، <code>Date</code> ، <code>TraineeId</code> ؛ <code>WorkoutPlanId</code> + <code>WorkoutDayId</code>)
└ WorkoutLogEntry	(<code>PRESCRIBED</code> ؛ الفعلي + لقطة الموصوف <code>Exercise</code>)
NutritionLog	(اختياري، <code>Date</code> ، <code>TraineeId</code> ؛ <code>NutritionPlanId</code>)
└ NutritionLogEntry	(<code>Food</code> ، <code>Quantity</code>)
ProgressEntry	(مستقل — ؛ وزن/دهون/مقاسات/تاريخ <code>TraineeId</code> ، جذر،)
TraineeNote	(مستقل — <code>CreatorId</code> = المؤلف؛ <code>Date</code> ، <code>Text</code> ، <code>TraineeId</code> ، جذر،)

الروابط عبر التجميعات مفاتيح **Guid فقط** — لا تنقل (navigation) عبر حدّ التجميعات.

`WorkoutExercise/WorkoutLogEntry/WorkoutTemplateExercise` → `Exercise`؛
`MealItem/NutritionLogEntry/NutritionTemplateItem` → `Food`؛
`Trainee` عبر `TraineeId`.

4.3 ملاحظات مفتاحية عن الكيانات

- **Trainee** — ملف تدريبي علاقته 1:1 مع `IdentityUser` من ABP (دور `Trainee`) عبر `UserId`.

بيانات الاعتماد على المستخدم، وبيانات التدريب هنا. `UserName` لقطة من اسم الدخول. `IsActive` يبدّلان `Deactivate()/Activate()`.

- `WorkoutDay.ScheduledDay` (`DayOfWeek?`) — يغذّي شاشة "اليوم". `null` = لا يظهر أبدًا كاليوم؛

يومٌ أسبوعي بلا يوم مجدول = يوم راحة.

- `WorkoutLogEntry` — يخزّن الفعلي (`Sets/Reps/WeightKg/Notes`) *و* لقطة الموصوف

(`PrescribedSets/PrescribedReps/PrescribedWeightKg`، تقبل `null`). تُلتقط اللقطة عند التسجيل من خطة فيبقى "المخطّط مقابل المنجز" حتى بعد تعديل الخطة. السجّلات اليدوية موصوفها `null`.

- `NutritionLogEntry/MealItem` يخزّنان فقط `FoodId` + `Quantity` (حصص). السرعات والماكروز

تُحسب من مكتبة `Food` وقت القراءة، لا تُخزّن على العنصر — فتعديل ماكروز طعام يصحّح كل شيء رجعيًا.

- القوالب تحاكي الخطط لكن تُسقط `TraineeId` و `IsActive` (لا مفهوم "مفعّل" لمخطّط)، وتعيد استخدام

`*Consts` الخاصة بالخطط (لا كلاسات ثوابت مستقلة).

4.4 التعدادات (`CoachApp.Enums`، تُخزّن ك `byte`)

- `Gender`: `Unspecified=0`, `Male=1`, `Female=2`
- `TrainingGoal`: `General=0`, `LoseWeight=1`, `BuildMuscle=2`, `Maintain=3`, `ImproveFitness=4`, `Strength=5`
- `MuscleGroup`: `Other=0`, `Chest=1`, `Back=2`, `Shoulders=3`, `Arms=4`, `Legs=5`, `Core=6`, `FullBody=7`, `Cardio=8`
- `Equipment`: `None=0`, `Bodyweight=1`, `Barbell=2`, `Dumbbell=3`, `Machine=4`, `Cable=5`, `Kettlebell=6`, `ResistanceBand=7`, `Other=8`
- إضافة إلى `System.DayOfWeek` (يستخدمه `ScheduledDay`).

4.5 الثوابت (حدود أطوال الحقول — `Domain.Shared/Entites/.../*Consts.cs`)

`TraineeConsts` (256 `UserName`، 64 `First/Last`، 256 `Email`، 32 `Phone`، 6–128 `Password`)،
`ExerciseConsts` (128 `Name`، 512 `Desc`، 2000 `Instructions`، 512 `MediaUrl`)، `FoodConsts` (128 `Name`،
512 `Desc`، 16 `ServingUnit`)، `WorkoutPlanConsts` (128 `Name`، 512 `Desc`، 64 `DayName`، 32 `Reps`)،
512 `Notes` — تستخدمها القوالب أيضًا)، `NutritionPlanConsts` (128 `Name`، 512 `Desc`، 64 `MealName`)،
— تستخدمها القوالب أيضًا)، `WorkoutLogConsts` (32 `Reps`، 512 `Notes`، **MaxEntries 100**)،
`NutritionLogConsts` (512 `Notes`، **MaxEntries 100**)، `ProgressEntryConsts` (512 `Notes`)،
`TraineeNoteConsts` (2000 `Text`).

5. التخزين (EF Core)

- `CoachAppDbContext` واحد (`/src/CoachApp.EntityFrameworkCore/EntityFrameworkCore/`).
يستبدل `DbContext` الخاصين بـ `Identity` و `TenantManagement` (`[ReplaceDbContext]`) لتتمكّن من الـ `JOIN` عبرها. اسم سلسلة الاتصال `Default`؛ بادئة الجداول `CoachAppConsts.DbTablePrefix = "App"`، مخطّط `null`.
- **21 DbSet** مجاليّاً (جذور + أبناء): `Trainees`، `Exercises`، `Foods`، `WorkoutDays`، `WorkoutPlans` / `WorkoutPlanTemplates`، `WorkoutTemplateDays`، `WorkoutExercises`، `WorkoutLogEntries`، `WorkoutLogs`، `WorkoutTemplateExercises`، `NutritionPlans`، `Meals`، `NutritionTemplateMeals`، `NutritionTemplateItems`، `NutritionPlanTemplates`، `MealItems`، `NutritionLogs`، `NutritionLogEntries`، `TraineeNotes`، `ProgressEntries`.
- لكل كيان `IEntityTypeConfiguration<T>` تحت `EntityConfigurations/<Feature>/`، وكلّها تُطبّق تلقائيّاً عبر `builder.ApplyConfigurationsFromAssembly(...).OnModelCreating`.
- الـ `migrations` في `CoachApp.EntityFrameworkCore/Migrations/`.

6. التحويل (Mapster)

- `CoachAppMapsterConfig.Configure()` — سطر `TypeAdapterConfig<Entity, >.NewConfig()` واحد
- لكل تحويل، كيان `→ DTO` فقط. الإنشاء/التحديث يمرّ عبر ميثودات سلوك التجميعية، لا عبر التحويل.
- `MapsterObjectMapper : IAutoObjectMappingProvider` — يمرّر `ObjectMapper.Map<...>()` عبر `Mapster`.
- مُسجّل في `CoachAppApplicationModule`.
- **الإثراء (Enrichment)** منفصل عن التحويل: قيم العرض عبر-التجميعية (أسماء التمارين والأطعمة، مجاميع الماكروز) تُملأ في الـ `AppService` عبر مساعدات مخصّصة — `WorkoutLogEnricher`، `NutritionPlanEnricher`، `NutritionLogEnricher`، `NutritionPlanTemplateEnricher` — تحمل صفوف المكتبة المرجعية دفعةً واحدة. حساب ماكروز الحصة معرّف في مكان واحد لكل مجال.

7. المصادقة والهوية وتعدّد المستأجرين

OpenIddict 7.1 (Domain/OpenIddict/OpenIddictDataSeedContributor.cs)

- نطاق (scope) API واحد: `CoachApp`.
 - عميلان (clients) (يُزرعان فقط إن كان ClientId مضبوطًا):
 - `CoachApp_App` — عميل عام يفعّل منح كلمة المرور (password grant / ROPC) إضافةً إلى
- هذا هو كيف يسجّل تطبيق الـ SPA/الموبايل دخول مدرّب أو متدرّب باسم مستخدم + كلمة مرور.
- `CoachApp_Swagger` — `authorization_code` فقط، لواجهة Swagger.
 - الاستضافة (`CoachAppHttpApiHostModule`): تحقّق (validation) OpenIddict بجمهور `CoachApp`، و `UseLocalServer()` — نفس العملية خادم مصادقة * و* خادم موارد. المطالبات الديناميكية مفعّلة (`IsDynamicClaimsEnabled = true`)، فتنعكس تغييرات الصلاحيات/الأدوار دون إعادة دخول. في الإنتاج يُحمّل `openiddict.pfx` حقيقي (عبارة المرور من `AuthServer:CertificatePassPhrase`).

7.2 زرع الأدوار والصلاحيات

- ثوابت أسماء الأدوار (`Domain.Shared/CoachAppRoles.cs`): `Coach`، `Trainee`.
 - بادئات الصلاحيات (`Domain.Shared/CoachAppPermissionPrefixes.cs`): `CoachApp.Coach`.
- `CoachApp.Trainee` (مكرّرة هنا لأن Domain لا يمكنه الإشارة إلى كلاس صلاحيات الـ Contracts).
- `Domain/Data/CoachAppDataSeedContributor.cs` يعمل للمضيف ولكل مستأجر، ولكلّ يضمن وجود دوري
- `Coach` و `Trainee` (بنطاق المستأجر) ويمنح **كامل شجرة `Coach.*` لـ `Coach` و `Coach` و `Trainee` ** كامل شجرة `Trainee.*` لـ `Trainee`.
- لا زرع مشرف مخصّص — المشرف الافتراضي من `IdentityDataSeedContributor` في ABP، بقيادة الافتراضي: `admin@abp.io` / `1q2w3E*` (`CoachAppConsts`). `CoachAppDbMigrationService.SeedDataAsync` (بريد/كلمة مرور المشرف عبر `DataSeedContext`).
 - يُنشأ مستأجر تجريبي ("demo") فقط عند `CoachApp:SeedDemoTenant == true` (مضبوط في `appsettings`)

الخاص بـ `DbMigrator`؛ يجب أن يكون `false`/غائبًا في الإنتاج).

7.3 تعدّد المستأجرين — "مستأجر لكل مدرّب"

- `MultiTenancyConsts.IsEnabled = true`؛ الاستضافة تنادي `app.UseMultiTenancy()`.
- كل مدرب = مستأجر واحد. مشرف المستأجر المزروع تلقائيًا * هو * المدرب. المتدربون مستخدمو `IdentityUser` إضافيون (دور `Trainee`) يُنشأون داخل ذلك المستأجر، كل مرتبط 1:1 بملف `Trainee` عبر `.UserId`.

- حارس المدرب الواحد لكل مستأجر: `Domain/Identity/SingleCoachPerTenantHandler.cs` — معالج حدث

محلي على إنشاء/تحديث `IdentityUser`. إن كان التغيير سيجعل مستخدمًا ثانيًا مدربًا في المستأجر، يرمي `BusinessException(SecondCoachNotAllowed)` ويتراجع عن وحدة العمل. يقصر المسار على مسارات الدخول/القفل/الملف الساخنة.

- المستأجر/المستخدم الحالي يحلّه ABP من التوكن في الـ pipeline؛ استخدم `ICurrentTenant` /

`ICurrentUser`. البذور تبدل السياق بـ `._currentTenant.Change(tenantId)`.

7.4 شجرة الصلاحيات (`Application.Contracts/Permissions/CoachAppPermissions.cs`)

مجموعة واحدة `CoachApp`. عُقد CRUD تُبنى بمساعد `AddCrud` (`.Create/.Update/.Delete`).
 شخصية المدرب (`CoachApp.Coach.*`)

العقدة	الأبناء
<code>Trainees</code>	<code>ResetPassword</code> , <code>Delete</code> , <code>Update</code> , <code>Create</code>
<code>WorkoutPlanTemplates</code> , <code>WorkoutPlans</code> , <code>Exercises</code> , <code>NutritionPlanTemplates</code> , <code>NutritionPlans</code> , <code>Foods</code>	<code>Delete</code> , <code>Update</code> , <code>Create</code>
<code>Notes</code> , <code>Progress</code>	<code>Delete</code> , <code>Update</code> , <code>Create</code>
<code>Tracking</code>	* (ورقة — قراءة فقط؛ مشتركة بين قراءات <code>NutritionLog</code> و <code>WorkoutLog</code> و <code>*(TraineeDashboard)</code>

شخصية المتدرب (`CoachApp.Trainee.*`)

العقدة	الأبناء
<code>MyWorkoutPlans</code> , <code>MyToday</code> , <code>MyDashboard</code> , <code>MyProfile</code> , <code>MyNotes</code> , <code>MyNutritionPlans</code>	* (ورقة)
<code>NutritionLogs</code> , <code>WorkoutLogs</code>	Create, Update, Delete — الحذف يعتمد على الورقة/Default)*
<code>MyProgress</code>	Create, Update, Delete — الحذف يعتمد على Default)*

عدم التماثل مقصود: للمدرب CRUD كامل على المكتبات/الخطط/المتدربين المشتركة؛ وللمتدرب قراءة + تسجيل إلحاق فقط (يسجل نشاطه لكن لا يحذف السجل).

7.5 رموز الأخطاء (`Domain.Shared/CoachAppDomainErrorCodes.cs`)

مربوطة بالترجمة عبر `MapCodeNamespace("CoachApp", typeof(CoachAppResource))`. تُرمى كـ `BusinessException(code)`؛ فيحصل العميل على رمز ثابت + رسالة مترجمة.

الرمز	الثابت	المعنى / مكان الفرض
<code>CoachApp:00001</code>	<code>TraineeNotFound</code>	الإشارة إلى متدرب غير موجود (خطة/قياس/ملاحظة/لوحة)
<code>CoachApp:00002</code>	<code>ExercisesNotFound</code>	تمرين مُشار إليه غير موجود
<code>CoachApp:00003</code>	<code>FoodsNotFound</code>	طعام مُشار إليه غير موجود
<code>CoachApp:00004</code>	<code>ExerciseInUse</code>	مُنِع الحذف — مستخدم في خطة/قالب/سجل (عُطِّل بدلاً منه)
<code>CoachApp:00005</code>	<code>FoodInUse</code>	مُنِع الحذف — مستخدم في خطة/قالب/سجل
<code>CoachApp:00006</code>	<code>DuplicateScheduledDay</code>	يوماً تمرين على نفس يوم الأسبوع

الرمز	الثابت	المعنى / مكان الفرض
CoachApp:00007	SecondCoachNotAllowed	مدرب واحد لكل مستأجر (حارس الهوية)
CoachApp:00008	CannotModifyCoachAuthoredProgress	لا يعدّل/يحذف المدرب إلا قياسات تقدّمه هو

الترجمة: **en.json** و **ar.json** هما الزوج المصان (يغطيان كليًا رموز الأخطاء + تسميات الصلاحيات + الأدوار + التعدادات). توجد بذور لغات أخرى لكنها غالبًا غير مترجمة.

8. خدمات جهة المدرب (My <Feature>/Apis، بلا بادئة My)

كلها تشتقّ من **CoachAppAppService**، مُصفاة بالمستأجر تلقائيًا، تحرس القراءة بـ **[Authorize(...Default)]** على مستوى الكلاس والكتابة بـ **[Authorize(...Create/Update/Delete)]** لكل ميثود. الاستعلامات مضمّنة (**Dynamic.Core + WhereIf + GetQueryableAsync + TraineedId**). **TraineedId** غير قابل للتغيير عند التحديث (لا يُعاد ربط خطة/قياس/ملاحظة بمتدرب آخر أبدًا).

الخدمة	جذر الصلاحية	أبرز النقاط
TraineedAppService	Coach.Trainees	تُنشئ الدخول + الملف (انظر 8.1). الحذف = تعطيل قابل للاسترجاع. .ResetPasswordAsync بحث/تصفية القائمة.
ExerciseAppService	Coach.Exercises	CRUD للمكتبة. يُمنع الحذف إن كان مُشارًا إليه (ExerciseInUse).
FoodAppService	Coach.Foods	CRUD للمكتبة (ماكروز للحصّة). يُمنع الحذف إن كان مُشارًا إليه (FoodInUse).

الخدمة	جذر الصلاحية	أبرز النقاط
WorkoutPlanAppService	Coach.WorkoutPlans	خطة -أيام- تمارين. تتحقق من وجود المتدرب + التمارين + تفرد يوم الأسبوع المجدول. <code>SetActiveAsync</code> تفرض خطة مفعلة واحدة لكل متدرب. استبدال كامل للبنية عند التحديث.
NutritionPlanAppService	Coach.NutritionPlans	خطة -وجبات- عناصر + أهداف اختيارية. تتحقق من المتدرب + الأطعمة. مفعلة واحدة لكل متدرب.
WorkoutPlanTemplateAppService	Coach.WorkoutPlanTemplates	CRUD للقوالب. → <code>CloneToTraineeAsync</code> WorkoutPlan غير مفعلة جديدة (بحارس <code>WorkoutPlans.CreateSaveAsTemplateAsync</code>). تلتقط خطة.
NutritionPlanTemplateAppService	Coach.NutritionPlanTemplates	نظير خدمة قالب التمرين (استنساخ/حفظ-كقالب، تنقل الأهداف).
WorkoutLogAppService	Coach.Tracking	قراءة فقط. عرض سجلات المتدرب بمدى زمني (مُثناة بأسماء التمارين).
NutritionLogAppService	Coach.Tracking	قراءة فقط. عرض سجلات تغذية المتدرب (مُثناة بالماكروز).
ProgressEntryAppService	Coach.Progress	CRUD كامل على خطّ قياسات جسم المتدرب.

الخدمة	جذر الصلاحية	أبرز النقاط
TraineeNoteAppService	<code>Coach.Notes</code>	CRUD على ملاحظات مؤرّخة لكل متدرّب.
TraineeDashboardAppService	<code>Coach.Tracking</code>	تحليلات قراءة فقط؛ تفوّض الحساب لـ <code>DashboardCalculator</code> .

8.1 كيف يُنشأ حساب دخول المتدرّب (`TraineeAppService.CreateAsync`)

يحقن `IRepository<Trainee, Guid>` + `IdentityUserManager` في نداء واحد:

1. إنشاء `IdentityUser` في `CurrentTenant.Id` (البريد افتراضياً `{UserName}@coachapp.local`).
2. `AddToRoleAsync(user, "Trainee") → CreateAsync(user, password)`. أخطاء الهوية تظهر كـ `UserFriendlyException` (عبر `CheckIdentityErrors`).

1. بناء الملف بـ `Trainee.Create(...)` مربوطًا بـ `user.Id`؛ ثم الإدراج.
2. `ApplyLoginStateAsync(user, IsActive)` — المفعل يزيل القفل؛ وغير المفعل يضع قفلاً دائماً.

فلا يستطيع المتدرّب الدخول. تُنادى من `Create/Update/Delete`.

`DeleteAsync` تعطيل قابل للاسترجاع (يُضبط `IsActive=false` + يقفل الدخول؛ يُحفظ السجل؛ إعادة التفعيل عبر `UpdateAsync`). `ResetPasswordAsync` تنقذ `AddPasswordAsync` + `RemovePasswordAsync` في وحدة عمل واحدة (بلا كلمة مرور حالية؛ اسم الدخول ثابت).

8.2 `DashboardCalculator` (مشارك، `Apis/Dashboards/DashboardCalculator.cs`، `internal`، `static`)

يستخدمه كلٌّ من لوحة المدرّب ولوحة المتدرّب (الفرق الوحيد: المدرّب يمرر معرّف متدرّب من الإدخال، والمتدرّب يحلّه من المستخدم الحالي).

- **الالتزام الغذائي (يوم):** جمع الماكروز المستهلكة عبر *كل* سجلّات ذلك اليوم؛ الهدف = أهداف الخطة

المفعّلة الصريحة، وإلاّ مجاميع الوجبات المحسوبة: `percent = consumed/target*100` (منزلة واحدة، بلا سقف، null إن لم تكن هناك خطة/هدف صفري). `OverallPercent = CaloriesPercent`.

- **الالتزام الغذائي (مدى):** `consumedTotal / (targetPerDay * daysLogged) * 100` — بمعدّل على الأيام

المسجلة فعليًا.

- إتمام التمارين: `CompletedSessions` = أيام متميزة فيها سجلّ ضمن المدى؛ `PlannedPerWeek` = أيام الأسبوع المجدولة المتميزة للخطّة المفعلّة؛ `Weeks = ceil(days/7)`؛ `CompletionPercent`` (بلا سقف، null إن لم تكن هناك خطّة مفعلّة). `completed/planned*100`

9. خدمات جهة المتدرب (/<Feature>Apis، ببادئة My)

9.1 قاعدة "من أنا" — `MyTraineeAppServiceBase`

`src/CoachApp.Application/MyTraineeAppServiceBase.cs` (يمدّد `CoachAppAppService`). المكان الوحيد الذي يُربط فيه المستخدم الداخل بمتدرب:

- `GetCurrentTraineeIdAsync()` / `GetCurrentTraineeAsync()` — يبحثان عن `Trainee` حيث `UserId == CurrentUser.GetId()`. لا صفّ `Trainee` → `EntityNotFoundException(typeof(TTrainee))`.
- ثابت أمني: معرّف المتدرب يُشتقّ دائمًا من المستخدم الحالي، لا من إدخال العميل أبدًا. لا يحمل أي DTO إنشاء/تحديث لجهة المتدرب `TraineeId`. الوصول لبيانات متدرب آخر يُعامل بانتظام كـ 404 (معرّف غريب → `EntityNotFoundException`) فلا تُفصح الملكية.

9.2 الخدمات

الخدمة	الصلاحية	ماذا تفعل
MyProfileAppService	<code>Trainee.MyProfile</code>	قراءة الملف الشخصي؛ تعديل ذاتي مقيد — فقط <code>PhoneNumber/Email/BirthDate</code> (الهدف/الأوزان/التفعيل/اسم الدخول تبقى للمتدرب).
MyWorkoutPlanAppService	<code>Trainee.MyWorkoutPlans</code>	قراءة فقط. <code>list/get/getActive</code> — الخطط المفعلّة فقط (غير المفعلّة مخفية). يُثري أسماء التمارين.
MyNutritionPlanAppService	<code>Trainee.MyNutritionPlans</code>	نظير قراءة فقط؛ مفعلّة فقط؛ يُثري الوجبات/الماكروز.

الخدمة	الصلاحيات	ماذا تفعل
MyWorkoutLogAppService	<code>Trainee.WorkoutLogs</code>	Create (يدوي) / CreateFromDay / list / delete / update / get. يتحقق من ملكية الخطة + اليوم والتمارين. يحفظ لقطة الموصوف عند التحديث.
MyNutritionLogAppService	<code>Trainee.NutritionLogs</code>	Create (يدوي) / CreateFromPlan / list / delete / update / get. يتحقق من ملكية الخطة والأطعمة.
MyProgressAppService	<code>Trainee.MyProgress</code>	إنشاء/تحديث/حذف قياساته + list/get. راية <code>IsCoachAuthored</code> ; لا يعدل ما ألفه المدرب <code>(CannotModifyCoachAuthoredProgress)</code> .
MyNoteAppService	<code>Trainee.MyNotes</code>	قراءة فقط — قراءة ملاحظات المدرب الموجهة لهذا المتدرب.
MyTodayAppService	<code>Trainee.MyToday</code>	شاشة "اليوم" المربّبة (انظر 9.3).
MyDashboardAppService	<code>Trainee.MyDashboard</code>	التزامه الغذائي (يوم + أسبوعي) وإتمام تمارينه، عبر <code>DashboardCalculator</code> .

9.3 شاشة "اليوم" (`MyTodayAppService.GetAsync`)

الإدخال يحمل `Date` اختياريًا بتوقيت العميل المحلي (لكي يحترم "اليوم" منطقة المتدرب الزمنية؛ null → `UtcNow.Date`). كل شيء يُحلّ للمتدرب الحالي. تركّب:

• التمرين: الخطة المفّلة → أيام اليوم المجدولة (`d.ScheduledDay <= Days.Where(d`)`)

`(date.DayOfWeek, IsRestDay)` \ (خطة مفّلة لكن لا شيء مجدول اليوم)، وأحدث سجلّ تمرين لليوم `(LatestWorkoutLog, AlreadyLoggedWorkoutToday)`.

• التغذية: الخطة المفّلة (وجبات + أهداف) + `NutritionAdherence` لليوم (عبر `DashboardCalculator`)

+ `AlreadyLoggedNutritionToday`

- يُقصر التاريخ إلى [2000-01-01, 2100-01-01] لإبقاء حساب مدى اليوم بعيدًا عن تجاوز `DateTime`.

9.4 تدفقات "التسجيل من الخطة" (لقطات من جهة الخادم، بصلاحية `Create`)

- `{WorkoutDayId, Date, Notes?}` — ال DTO هو فقط `CreateFromDayAsync(CreateWorkoutLogFromDayDto)`
- تحلّ الملكية عبر جذر الخطة (`TraineeId == me && Days.Any(d => d.Id == WorkoutDayId)`؛ الغريب 404، ومعرّف الأب لا يُفصح). كل تمرين موصوف يصبح مُدخل سجلّ يُبذّر `**الفعلي` فيه من، ولقطة الموصوف تساوي، `Sets/Reps/WeightKg` الخاصّة بالخطة `**`.
- `{NutritionPlanId, Date, Notes?}` — `CreateFromPlanAsync(CreateNutritionLogFromPlanDto)` فحص
- ملكية، ثم تفريد الوجبات → العناصر إلى مُدخلات مرتّبة (`Quantity + FoodId`).
- حفظ لقطة الموصوف عند التحديث: `UpdateAsync` يسمح المُدخلات ويعيد بناءها من إدخال العميل، لكن
- قيم الموصوف تُلتقط مسبقًا بمفتاح (`ExerciseId, Order`) وتُستعاد — لا يستطيع إدخال العميل الكتابة فوق سجلّ الوصفة أبدًا.

10. قواعد شاملة ومكان فرضها

القاعدة	تُفرض في
عزل المستأجر (المدرّب يرى بياناته فقط)	مُصقّي المستأجر في ABP على كل جذر <code>IMultiTenant</code>
مدرّب واحد لكل مستأجر	<code>SingleCoachPerTenantHandler</code> (حدث هوية) → <code>SecondCoachNotAllowed</code>
خطة تمرين/تغذية مفعّلة واحدة لكل متدرّب	خدمات الخطط (<code>SetActiveInternalAsync</code>)
تمرين واحد مجدول كحدّ أقصى لكل يوم أسبوع	→ <code>WorkoutPlanAppService.ValidateScheduledDaysUniqueDuplicateScheduledDay</code>

القاعدة	تُفَرَضُ فِي
وجوب وجود المتدرب/التمرين/الطعام المُشار إليه	فحوص الوجود في الخدمة → <code>TraineeNotFound/ExercisesNotFound/FoodsNotFound</code>
منع حذف عنصر مكتبة مستخدم	حراس حذف <code>ExerciseInUse/FoodInUse</code> → <code>Exercise/Food</code>
المتدرب يمسّ بياناته فقط	<code>MyTraineeAppServiceBase</code> + تصفية <code>TraineeId==me</code> (الغريب → 404)
المتدرب لا يعدّل قياسات ألفها المدرّب	→ <code>MyProgressAppService.EnsureSelfAuthored CannotModifyCoachAuthoredProgress</code>
<code>TraineeId</code> غير قابل للتغيير عند التحديث	ميثودات تحديث جهة المدرّب تتجاهل <code>TraineeId</code> الوارد
ديمومة لقطة الموصوف-مقابل- الفعلي	مُدخلات <code>WorkoutLog</code> + استعادة-بالمفتاح في تحديث <code>MyWorkoutLog</code>
أطوال الحقول / الحقول المطلوبة	<code>*Check.*</code> في الكيان مقابل <code>*Consts</code>

11. البناء والتشغيل وال Migration والاختبار

```
# بناء الحلّ كاملاً (يجب أن يكون نظيفاً)
dotnet build CoachApp.slnx

# تشغيل الـ API + Swagger
dotnet run --project src/CoachApp.HttpApi.Host # https://localhost:44370/swagger

# الاختبارات (xUnit + Shouldly)
dotnet test

# (كتابة الملفات مسموحة دائماً) migration إضافة
dotnet ef migrations add <Name> \
  --project src/CoachApp.EntityFrameworkCore \
  --startup-project src/CoachApp.EntityFrameworkCore
```

```
# زرع الأدوار/الصلاحيات/المشرف (⚠️ يمسّ قاعدة البيانات) + migrations تطبيق الـ
dotnet run --project src/CoachApp.DbMigrator
```

⚠️ تطبيق migration أو تشغيل DbMigrator يمسّ قاعدة البيانات — أكّد دائماً مع الفريق قبل ذلك.
توليد/كتابة ملفات ال migration مسموح؛ تنفيذها ليس تلقائياً.

سلسلة اتصال التطوير الافتراضية (HttpApi.Host/appsettings.json): LocalDB، قاعدة CoachApp. سلطة المصادقة https://localhost:44370، عميل Swagger CoachApp_Swagger.

12. إضافة ميزة (شريحة عمودية، من الأسفل للأعلى)

احتذ شريحة Trainee / WorkoutPlan:

1. Domain.Shared → Entites/<Feature>/<Entity>Consts.cs، أي تعداد في Enums/، رمز خطأ في

CoachAppDomainErrorCodes.

1. Domain → Entites/<Feature>/<Entity>.cs (جذر: FullAuditedAggregateRoot<Guid> + IMultiTenant،

static Create، ميثودات سلوك؛ الأبناء عبر Add*).

1. Application.Contracts → Entites/<Feature>/ DTOs FullAuditedEntityDto<Guid> للخروج،

Create/Update للدخل، (Get...ListInput : PagedAndSortedResultRequestDto + I<Entity>AppService +

1. Application.Contracts/Permissions → أضف عُقداً إلى CoachAppPermissions +

CoachAppPermissionDefinitionProvider (شجرة Coach و/أو Trainee؛ البذرة تمنحها تلقائياً).

1. Application → Apis/<Feature>/<Entity>AppService.cs (سمينة، استعلامات مضمّنة، [Authorize]

فحوص وجود/فرادة) + سطر واحد في CoachAppMapsterConfig (+ enricher إن احتاج بيانات عرض عبر-تجميعية). لجهة المتدرب؟ امُدّد MyTraineeAppServiceBase.

1. EntityFrameworkCore → EntityConfigurations/<Feature>/<Entity>Configuration.cs، أضف DbSet،

dotnet ef migrations add

1. الترجمة → مفاتيح في كلّ من en.json و ar.json.

2. الاختبارات → احتد `test/CoachApp.*.Tests`.

القائمة التفصيلية والوكلاء المتخصصون في `./claude/agents/` (`coachapp-architect`، `-backend`، `-db`، `-tester`، `-reviewer`، `-security`، `-runner`) والقواعد في `./cursor/rules/**/*.mdc`.

13. قرارات تصميمية بارزة وبنود مفتوحة

- **الحذف = تعطيل للمتدربين وعناصر المكتبة (قابل للاسترجاع؛ السجلّ محفوظ).** المسح النهائي مؤجل (مُشار إليه V1.1).

- **المتدرب يرى الخطط المفعّلة فقط** — غير المفعّلة/المؤرشفة/المسوّدة مخفية عن جهة المتدرب (مؤجل)

إلى V1.1).

- **النسب بلا سقف** في اللوحات — تجاوز الهدف يبقى ظاهرًا (لا يُقصر إلى 100%).
- **يُسمح بعدّة سجلّات في اليوم؛** شاشة اليوم تُظهر الأحداث، واللوحات تجمعها.
- **ماكروز التغذية تُحسب دائمًا من مكتبة الأطعمة وقت القراءة** — لا تُنسخ على عناصر الخطّة/السجلّ

فتصحیح طعام ینتشر فی کل مکان.

- **الموصوف مقابل الفعلي** مفهوم من الدرجة الأولى على سجلّات التمرين فقط؛ "من الخطّة" في التغذية

يصبح ببساطة المُدخّلات الفعلية (بلا مفهوم موصوف).

- **ar + en** هما الترجمتان المُصانّتان بالكامل؛ بقية بذور اللغات غير مترجمة غالبًا.
- **كتابة Entites** (المفارقة الإملائية) حاملة للبنية عبر فضاءات الأسماء والمجلّدات — أبقيها.